

Healthcare Management Ecosystem

Client Test Guide — deployed environment walkthrough

Purpose. This guide walks through a full, realistic use of the system as deployed today: creating an account, setting up a hospital's structure, registering a patient, and taking one appointment all the way from booking through completion. No technical setup is required — every step is done in a web browser.

Environment	Live deployment (Render, free tier)
Base URL	<code>https://healthcare-app-vtwa.onrender.com</code>
What you need	A web browser. Nothing else to install.
Estimated time	20–30 minutes

Contents:

- 1. Before you start — the four test consoles
- 2. Create an account and sign in
- 3. Set up the hospital structure
- 4. Add a doctor and their weekly schedule
- 5. Register a patient
- 6. Generate bookable appointment slots
- 7. Book an appointment
- 8. Walk the appointment through check-in and completion
- 9. Optional: duplicate-patient detection
- 10. Result checklist
- 11. What to expect / known limitations

1. Before you start — the four test consoles

Every step below is done through one of these four pages. Each is a live, interactive form: you fill in a box, click a button, and see the real response. Open all four in separate browser tabs now:

Console	Link	Used for
Identity	<code>/docs/identity</code>	Creating your account, signing in

Facility	<code>/docs/facility</code>	Hospital, departments, doctor schedules
Patient	<code>/docs/patient</code>	Registering patients
Scheduling	<code>/docs/scheduling</code>	Appointment slots, booking, queue

(Full addresses: prefix each with `https://healthcare-app-vtwa.onrender.com`.)

Tip: On each page, click any endpoint to expand it, then click the **"Try it out"** button, edit the example JSON in the box, and click **"Execute"**. The response appears right below, with its status code.

2. Create an account and sign in

Goal: confirm you can register a user and log in.

2a. Register

On the **Identity** console, open **POST** `/auth/register`.

```
{
  "email": "your.name@example.com",
  "password": "Password123!",
  "fullName": "Your Name"
}
```

Expect: status **201**, with a response containing an `id` and `"status": "pending_verification"`.

2b. Sign in

Open **POST** `/auth/login` with the same email/password.

Expect: status **201**, with an `accessToken`, `refreshToken`, and `expiresIn`. You won't need to use the token for the rest of this guide — every action below works without it on this test deployment (see section 11).

3. Set up the hospital structure

Goal: create an organization, one facility under it, and one department.

3a. Create the organization

On the **Facility** console: **POST** `/organizations`

```
{
  "legalName": "Black Lion General Hospital PLC",
  "displayName": "Black Lion Hospital",
  "type": "hospital"
}
```

Expect: 201, with an `id` — copy it, you'll need it next. (Valid `type` values: `healthcare_network`, `hospital`, `clinic`, `pharmacy_chain`, `diagnostic_center`.)

3b. Create a facility under it

POST `/organizations/{organizationId}/facilities` — paste the organization's `id` into the `organizationId` field on the page.

```
{
  "name": "Black Lion Hospital - Main Campus",
  "type": "hospital"
}
```

Expect: 201, with a facility `id`. Copy it too.

3c. Create a department

POST `/facilities/{facilityId}/departments`

```
{
  "name": "Internal Medicine",
  "type": "outpatient"
}
```

Expect: 201, with a department `id`. Copy it — you'll use it repeatedly from here on.

4. Add a doctor and their weekly schedule

Goal: assign the user you registered in step 2 as a doctor, and give them working hours.

Go back to your **Identity** registration response from step 2a and copy that user's `id` — that's the doctor's id for everything below.

4a. Assign the doctor to the department

POST `/departments/{id}/providers` (use the department id from 3c)

```
{
  "providerId": "<the user id from step 2a>",
  "roleTitle": "Attending Physician"
}
```

Expect: 201.

4b. Give them a weekly schedule

POST `/departments/{id}/schedules`

```
{
  "providerId": "<the user id from step 2a>",
  "dayOfWeek": 2,
  "startTime": "08:00",
  "endTime": "16:00"
}
```

Expect: 201. `dayOfWeek` is a number, 0 = Sunday through 6 = Saturday (2 = Tuesday). Times are 24-hour `HH:mm`. This creates a *recurring* weekly template — no bookable appointment exists yet; that's step 6.

5. Register a patient

Goal: register a patient who will book the appointment.

On the **Patient** console: **POST** /patients

```
{
  "firstName": "Sara",
  "lastName": "Tesfaye",
  "dateOfBirth": "1990-05-14",
  "gender": "female",
  "nationalId": "ETH-99991234"
}
```

Expect: 201, with a patient `id` and `"possibleDuplicates": []` (empty, since this is a new person). Copy the patient `id`.

6. Generate bookable appointment slots

Goal: turn the doctor's weekly schedule from step 4b into real, dated appointment slots someone can actually book.

On the **Scheduling** console, first create an appointment type: **POST** `/appointment-types`

```
{
  "facilityId": "<facility id from step 3b>",
  "name": "General Consultation",
  "code": "GEN_CONSULT",
  "defaultDurationMins": 30
}
```

Expect: 201, with an appointment-type `id`.

6a. Generate the slots

POST `/schedule-slots/generate`

```
{
  "departmentId": "<department id from step 3c>",
  "providerId": "<doctor's user id from step 2a>",
  "dateFrom": "<an upcoming Monday's date, e.g. 2026-11-02>",
  "dateTo": "<7 days later>",
  "slotDurationMins": 30
}
```

Important: the date range you choose must include at least one actual Tuesday (since step 4b set `dayOfWeek: 2`), or nothing will be generated. Pick any real week that contains a Tuesday.

Expect: 201, with `"createdCount"` greater than 0 (16 slots for an 08:00–16:00 day in 30-minute blocks) and a list of the created slots.

6b. Confirm the slots are listed as open

GET `/schedule-slots?providerId=<doctor id>&status=open`

Expect: 200, a list of open slots. Copy the `id` of the first one.

7. Book an appointment

Goal: book the patient from step 5 into the slot from step 6b.

POST /appointments

```
{
  "patientId": "<patient id from step 5>",
  "slotId": "<slot id from step 6b>",
  "appointmentTypeId": "<appointment-type id from step 6>",
  "reason": "Routine checkup"
}
```

Expect: 201, with the new appointment's `id` and `"status": "requested"`. Copy the appointment `id` — you'll use it for every remaining step. If you re-run this exact request a second time with the same slot, it correctly fails (the slot is no longer open) — that double-booking protection is expected behavior, worth trying once.

8. Walk the appointment through check-in and completion

Goal: move the appointment through its full real-world lifecycle and see the patient appear on, then clear from, the department's live queue.

1. **POST** `/appointments/{id}/confirm` — expect 201, status becomes `confirmed`.
2. **POST** `/appointments/{id}/check-in` — expect 201, status becomes `checked_in`. This also puts the patient on the department's queue.
3. On the **Scheduling** console: **GET** `/queue?facilityId=<facility id>&departmentId=<department id>` — expect one entry, `"status": "waiting"`, `"queueNumber": 1`. Copy that queue entry's `id`.
4. **POST** `/queue/{id}/call` — announces the patient's number; status becomes `called`.
5. **POST** `/appointments/{id}/start` — status becomes `in_service`.
6. **POST** `/appointments/{id}/complete` — status becomes `completed`.
7. Repeat the queue lookup from step 3 above — expect an **empty list** now, since completed visits drop off the active queue.

Worth trying: call `/confirm` a second time on the now-completed appointment. It should correctly fail — the system won't let an appointment go backwards or repeat a step once it's finished.

9. Optional: duplicate-patient detection

Goal: see how the system flags — but never silently merges — a possible duplicate patient.

Repeat step 5 exactly, using the *same* `nationalId` ("ETH-99991234") but any other name.

Expect: 201 — a new patient record is still created (it never blocks registration) — but this time `"possibleDuplicates"` is not empty, showing a high-confidence match against the first patient. Merging two records always requires a separate, explicit human decision (a `/merge-cases/.../confirm` call) — it never happens automatically.

10. Result checklist

Step	Pass	Fail	Notes
2. Register & log in	☐	☐	
3. Hospital structure created	☐	☐	
4. Doctor + schedule added	☐	☐	
5. Patient registered	☐	☐	
6. Slots generated	☐	☐	
7. Appointment booked	☐	☐	
8. Full lifecycle + queue	☐	☐	
9. Duplicate detection (optional)	☐	☐	

11. What to expect / known limitations

This is a free-tier test deployment, not production infrastructure:

- If the site has been idle for a while, the *first* request can take 15–30 seconds while it wakes back up. This is normal — just wait and retry.
- Login tokens are not currently required to perform actions in this test environment — every step above will work whether or not you're signed in. This is a known, intentional gap for this stage of testing, not a bug.
- There's no messaging/notification delivery on this deployment (no SMS or email will actually go out) — reminders and events are recorded internally only.

Healthcare Management Ecosystem — client test guide. Generated for the deployment at <https://healthcare-app-vtwa.onrender.com>. Questions or issues found while testing should be reported back with the step number, the request you sent, and the exact response you received.